Übungsblatt 9

Java Web Stack, Maven & Web-Architekturmuster

5430UE Web and Data Engineering

17. Juni 2026

Prof. Dr. Michael Granitzer, Tobias Fuchs



Lernziele

- Grundlegendes Verständnis des Java Web Stacks und des Build-Managements mit Maven.
- Einblick in die serverseitige Implementierung von REST-Schnittstellen mit Spring Boot.
- Grundlegendes Verständnis von Web-Architekturmustern und Session-Tracking-Methoden.



Java Web Stack Begriffe



Java Web Stack Begriffe (1/2)

a) Gegeben sind folgende Begriffe:
View Template Engine, Application Server, Build Management Tool, Test Framework, CI/CD Tool, Datenbank, Web Framework

Erläutern Sie diese jeweils kurz in einem Satz.



Java Web Stack Begriffe (2/2)

b) Ordnen Sie die folgenden Technologien der richtigen Kategorie (Begriffe aus a)) im Java Web Stack zu und geben Sie eine kurze Beschreibung ihrer Aufgabe:

Technologie	Kategorie	Kurzbeschreibung
Thymeleaf	?	?
Tomcat	?	?
Maven	?	?
JUnit	?	?
Jenkins	?	?
H2	?	?
Spring Framework	?	?

Java Web Stack Begriffe — Lösung (1/3)

1. Erklärung der Begriffe:
   - **View Template Engine**: Eine View Template Engine ist ein Werkzeug, das Daten in vorgefertigte HTML-Vorlagen einfügt, um daraus dynamisch Webseiten zu erzeugen.
   - **Application Server**: Ein Application Server ist eine Software, die Anwendungen ausführt und Anfragen von Nutzern entgegennimmt und verarbeitet.
   - **Build Management Tool**: Ein Build Management Tool automatisiert das Kompilieren, Testen und Verpacken eines Programms (u.a. Verwaltung von Abhängigkeiten (Dependency Graphs & Repositories)).
   - **Test Framework**: Ein Test Framework stellt eine Struktur und Werkzeuge bereit, um Software automatisch auf Fehler zu überprüfen.
   - **CI/CD Tool**: Ein CI/CD Tool automatisiert das kontinuierliche Integrieren von Codeänderungen sowie das Testen und Bereitstellen der Anwendung.
   - **Datenbank**: Eine Datenbank ist ein System zur strukturierten Speicherung und schnellen Abfrage von Daten.
   - **Web Framework**: Ein Web Framework ist eine Sammlung von Werkzeugen und Vorgaben, die die Entwicklung von Webanwendungen erleichtern.

Java Web Stack Begriffe — Lösung (2/3)

2. Zuordnung (Teil 1):

Technologie	Kategorie	Kurzbeschreibung
Thymeleaf		
Tomcat		
Maven		
JUnit		

Java Web Stack Begriffe — Lösung (2/3)

2. Zuordnung (Teil 1):

Technologie	Kategorie	Kurzbeschreibung
Thymeleaf	View Template Engine	
Tomcat		
Maven		
JUnit		

Java Web Stack Begriffe — Lösung (2/3)

2. Zuordnung (Teil 1):

Technologie	Kategorie	Kurzbeschreibung
Thymeleaf	View Template Engine	Server-seitiges HTML-Templating; Verarbeitet ein Template und ersetzt darin enthaltene Platzhalter, bevor die Seite angezeigt; Templates werden im Browser ohne Server darstellbar
Tomcat		
Maven		
JUnit		

Java Web Stack Begriffe — Lösung (2/3)

2. Zuordnung (Teil 1):

Technologie	Kategorie	Kurzbeschreibung
Thymeleaf	View Template Engine	Server-seitiges HTML-Templating; Verarbeitet ein Template und ersetzt darin enthaltene Platzhalter, bevor die Seite angezeigt; Templates werden im Browser ohne Server darstellbar
Tomcat	Application Server	
Maven		
JUnit		

Java Web Stack Begriffe — Lösung (2/3)

2. Zuordnung (Teil 1):

Technologie	Kategorie	Kurzbeschreibung
Thymeleaf	View Template Engine	Server-seitiges HTML-Templating; Verarbeitet ein Template und ersetzt darin enthaltene Platzhalter, bevor die Seite angezeigt; Templates werden im Browser ohne Server darstellbar
Tomcat	Application Server	Servlet-Container, der HTTP-Anfragen entgegennimmt und Java-Webapplikationen ausführt
Maven		
JUnit		

Java Web Stack Begriffe — Lösung (2/3)

2. Zuordnung (Teil 1):

Technologie	Kategorie	Kurzbeschreibung
Thymeleaf	View Template Engine	Server-seitiges HTML-Templating; Verarbeitet ein Template und ersetzt darin enthaltene Platzhalter, bevor die Seite angezeigt; Templates werden im Browser ohne Server darstellbar
Tomcat	Application Server	Servlet-Container, der HTTP-Anfragen entgegennimmt und Java-Webapplikationen ausführt
Maven	Build Management Tool	
JUnit		

Java Web Stack Begriffe — Lösung (2/3)

2. Zuordnung (Teil 1):

Technologie	Kategorie	Kurzbeschreibung
Thymeleaf	View Template Engine	Server-seitiges HTML-Templating; Verarbeitet ein Template und ersetzt darin enthaltene Platzhalter, bevor die Seite angezeigt; Templates werden im Browser ohne Server darstellbar
Tomcat	Application Server	Servlet-Container, der HTTP-Anfragen entgegennimmt und Java-Webapplikationen ausführt
Maven	Build Management Tool	Verwaltet Abhängigkeiten, steuert den Build-Lifecycle (compile, test, package, deploy) per `pom.xml`
JUnit		

Java Web Stack Begriffe — Lösung (2/3)

2. Zuordnung (Teil 1):

Technologie	Kategorie	Kurzbeschreibung
Thymeleaf	View Template Engine	Server-seitiges HTML-Templating; Verarbeitet ein Template und ersetzt darin enthaltene Platzhalter, bevor die Seite angezeigt; Templates werden im Browser ohne Server darstellbar
Tomcat	Application Server	Servlet-Container, der HTTP-Anfragen entgegennimmt und Java-Webapplikationen ausführt
Maven	Build Management Tool	Verwaltet Abhängigkeiten, steuert den Build-Lifecycle (compile, test, package, deploy) per `pom.xml`
JUnit	Test Framework	

Java Web Stack Begriffe — Lösung (2/3)

2. Zuordnung (Teil 1):

Technologie	Kategorie	Kurzbeschreibung
Thymeleaf	View Template Engine	Server-seitiges HTML-Templating; Verarbeitet ein Template und ersetzt darin enthaltene Platzhalter, bevor die Seite angezeigt; Templates werden im Browser ohne Server darstellbar
Tomcat	Application Server	Servlet-Container, der HTTP-Anfragen entgegennimmt und Java-Webapplikationen ausführt
Maven	Build Management Tool	Verwaltet Abhängigkeiten, steuert den Build-Lifecycle (compile, test, package, deploy) per `pom.xml`
JUnit	Test Framework	Standard-Framework für Unit- und Integrationstests in Java; Annotationen wie `@Test`

Java Web Stack Begriffe — Lösung (3/3)

3. Zuordnung (Teil 2):

Technologie	Kategorie	Kurzbeschreibung
Jenkins		
H2		
Spring Framework		

Java Web Stack Begriffe — Lösung (3/3)

3. Zuordnung (Teil 2):

Technologie	Kategorie	Kurzbeschreibung
Jenkins	CI/CD Tool	
H2		
Spring Framework		

Java Web Stack Begriffe — Lösung (3/3)

3. Zuordnung (Teil 2):

Technologie	Kategorie	Kurzbeschreibung
Jenkins	CI/CD Tool	Jenkins ist ein Automatisierungsserver, der Programmcode automatisch zusammenführt, testet und die fertige Software bereitstellt; er führt sogenannte „Pipelines“ aus, also vordefinierte Abläufe von Arbeitsschritten.
H2		
Spring Framework		

Java Web Stack Begriffe — Lösung (3/3)

3. Zuordnung (Teil 2):

Technologie	Kategorie	Kurzbeschreibung
Jenkins	CI/CD Tool	Jenkins ist ein Automatisierungsserver, der Programmcode automatisch zusammenführt, testet und die fertige Software bereitstellt; er führt sogenannte „Pipelines“ aus, also vordefinierte Abläufe von Arbeitsschritten.
H2	Datenbank	
Spring Framework		

Java Web Stack Begriffe — Lösung (3/3)

3. Zuordnung (Teil 2):

Technologie	Kategorie	Kurzbeschreibung
Jenkins	CI/CD Tool	Jenkins ist ein Automatisierungsserver, der Programmcode automatisch zusammenführt, testet und die fertige Software bereitstellt; er führt sogenannte „Pipelines“ aus, also vordefinierte Abläufe von Arbeitsschritten.
H2	Datenbank	Leichtgewichtige In-Memory-Datenbank für Entwicklung und Tests; vollständige JDBC-/JPA-Unterstützung
Spring Framework		

Java Web Stack Begriffe — Lösung (3/3)

3. Zuordnung (Teil 2):

Technologie	Kategorie	Kurzbeschreibung
Jenkins	CI/CD Tool	Jenkins ist ein Automatisierungsserver, der Programmcode automatisch zusammenführt, testet und die fertige Software bereitstellt; er führt sogenannte „Pipelines“ aus, also vordefinierte Abläufe von Arbeitsschritten.
H2	Datenbank	Leichtgewichtige In-Memory-Datenbank für Entwicklung und Tests; vollständige JDBC-/JPA-Unterstützung
Spring Framework	Web Framework	

Java Web Stack Begriffe — Lösung (3/3)

3. Zuordnung (Teil 2):

Technologie	Kategorie	Kurzbeschreibung
Jenkins	CI/CD Tool	Jenkins ist ein Automatisierungsserver, der Programmcode automatisch zusammenführt, testet und die fertige Software bereitstellt; er führt sogenannte „Pipelines“ aus, also vordefinierte Abläufe von Arbeitsschritten.
H2	Datenbank	Leichtgewichtige In-Memory-Datenbank für Entwicklung und Tests; vollständige JDBC-/JPA-Unterstützung
Spring Framework	Web Framework	Umfassendes Java-Framework: IoC-Container, MVC, Security, Data u.v.m.; Basis für Spring Boot

Maven Grundlagen



Maven Grundlagen (1/2)

Machen Sie sich auf maven.apache.org/what-is-maven.html mit den Grundideen von Apache Maven vertraut.

1. Was ist Maven grundlegend und welche Kernprobleme der Softwareentwicklung löst es?
2. Was ist ein Maven-Artefakt? Wie ist eine Maven-Koordinate aufgebaut? Geben Sie ein konkretes Beispiel und erklären Sie die einzelnen Bestandteile.
3. Was bedeutet semantische Versionierung? Erklären Sie `MAJOR.MINOR.PATCH` anhand eines Beispiels.
4. Was versteht man unter einem Build-Lebenszyklus in Maven und warum ist dieses Konzept für Entwickler nützlich? Nennen Sie zudem mindestens vier Phasen des *Maven Default Lifecycle* in der richtigen Reihenfolge.

Maven Grundlagen — Lösung (1/5)

1. **Was ist Maven?**

   Apache Maven ist ein umfassendes Werkzeug zum Build-Management und zur Projektverwaltung für Java-Anwendungen. Es basiert auf dem Konzept des *Project Object Model (POM)*, welches deklarativ in der Datei `pom.xml` beschrieben wird.

   Maven löst im Wesentlichen zwei Kernprobleme:

   - **Automatisierung des Build-Prozesses**: Statt Quellcode, Ressourcen, Tests und Paketierung manuell oder über komplexe, individuelle Skripte zu steuern, bietet Maven einen standardisierten, reproduzierbaren Ablauf.
   - **Dependency Management**: Moderne Software nutzt hunderte externe Bibliotheken (Libraries). Maven lädt diese Abhängigkeiten automatisch aus zentralen Online-Verzeichnissen (z. B. Maven Central) herunter und bindet sie in den Klassenpfad ein. Entwickler müssen JAR-Dateien somit nicht mehr manuell suchen und im Projekt abspeichern.



Maven Grundlagen — Lösung (2/5)

2. **Maven-Artefakt und Koordinaten**:
   Ein *Maven-Artefakt* ist das finale, kompilierte Ergebnis eines Build-Prozesses, das in einem Repository bereitgestellt wird (z. B. eine `.jar`-Bibliothek oder eine ausführbare `.war`-Webanwendung).

   Damit Maven Bibliotheken eindeutig identifizieren und herunterladen kann, besitzt jedes Artefakt eine eindeutige, dreiteilige *Koordinate*:

```
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
<version>3.5.0</version>
```

   Erklärung der Bestandteile:

   - `groupId`: Identifiziert die Organisation, das Unternehmen oder das Open-Source-Projekt. Sie folgt meist den Namenskonventionen von Java-Paketen (oft die umgedrehte Domain, z. B. `org.springframework`).
   - `artifactId`: Der konkrete, eindeutige Name des Moduls oder der Bibliothek innerhalb dieser Gruppe.
   - `version`: Die exakte Versionsnummer des Artefakts, um sicherzustellen, dass immer der exakt gleiche Code verwendet wird.

Maven Grundlagen — Lösung (3/5)

3. **Semantische Versionierung (Semantic Versioning)**:
   Semantische Versionierung ist ein Standard für den Aufbau von Versionsnummern. Eine Versionsnummer besteht aus drei durch Punkte getrennten Zahlen im Format `MAJOR.MINOR.PATCH` (z. B. `2.13.4`).

   Grundregel: Solange `MAJOR = 0`, befindet sich die Software in der Anfangsentwicklung und die API ist instabil. Ab Version `1.0.0` gelten folgende feste Versprechen zur Abwärtskompatibilität:

Komponente	Bedeutung	Beispiel-Wechsel
MAJOR	Inkompatible Änderung der API. Bestehender Code bricht beim Update u. U. ab.	`2.5.1` → `3.0.0`
MINOR	Neue Funktionalität wird hinzugefügt. Vollständig abwärtskompatibel.	`3.1.4` → `3.2.0`
PATCH	Reine Fehlerbehebungen (Bugfixes). Vollständig abwärtskompatibel.	`3.2.3` → `3.2.4`

Maven Grundlagen — Lösung (4/5)

4. **Der Maven Build-Lebenszyklus (Build Lifecycle)**:
   Ein Build-Lebenszyklus ist eine feste, vordefinierte Abfolge von einzelnen *Phasen* (Build Phases). Das Konzept sorgt dafür, dass jeder Entwickler und jedes automatisierte System (z. B. CI/CD-Pipelines) ein Projekt auf exakt dieselbe Weise baut.
   Die Phasen bauen aufeinander auf. Ruft man eine Phase auf, führt Maven automatisch alle vorhergehenden Phasen aus.

Phase	Fachliche Bedeutung und Aufgabe

Maven Grundlagen — Lösung (4/5)

4. **Der Maven Build-Lebenszyklus (Build Lifecycle)**:
   Ein Build-Lebenszyklus ist eine feste, vordefinierte Abfolge von einzelnen *Phasen* (Build Phases). Das Konzept sorgt dafür, dass jeder Entwickler und jedes automatisierte System (z. B. CI/CD-Pipelines) ein Projekt auf exakt dieselbe Weise baut.
   Die Phasen bauen aufeinander auf. Ruft man eine Phase auf, führt Maven automatisch alle vorhergehenden Phasen aus.

Phase	Fachliche Bedeutung und Aufgabe
`validate`	

Maven Grundlagen — Lösung (4/5)

4. **Der Maven Build-Lebenszyklus (Build Lifecycle)**:
   Ein Build-Lebenszyklus ist eine feste, vordefinierte Abfolge von einzelnen *Phasen* (Build Phases). Das Konzept sorgt dafür, dass jeder Entwickler und jedes automatisierte System (z. B. CI/CD-Pipelines) ein Projekt auf exakt dieselbe Weise baut.
   Die Phasen bauen aufeinander auf. Ruft man eine Phase auf, führt Maven automatisch alle vorhergehenden Phasen aus.

Phase	Fachliche Bedeutung und Aufgabe
`validate`	Prüft, ob das Projekt syntaktisch korrekt konfiguriert ist und alle notwendigen Informationen (z. B. in der `pom.xml`) vorhanden sind.

Maven Grundlagen — Lösung (4/5)

4. **Der Maven Build-Lebenszyklus (Build Lifecycle)**:
   Ein Build-Lebenszyklus ist eine feste, vordefinierte Abfolge von einzelnen *Phasen* (Build Phases). Das Konzept sorgt dafür, dass jeder Entwickler und jedes automatisierte System (z. B. CI/CD-Pipelines) ein Projekt auf exakt dieselbe Weise baut.
   Die Phasen bauen aufeinander auf. Ruft man eine Phase auf, führt Maven automatisch alle vorhergehenden Phasen aus.

Phase	Fachliche Bedeutung und Aufgabe
`validate`	Prüft, ob das Projekt syntaktisch korrekt konfiguriert ist und alle notwendigen Informationen (z. B. in der `pom.xml`) vorhanden sind.
`compile`	

Maven Grundlagen — Lösung (4/5)

4. **Der Maven Build-Lebenszyklus (Build Lifecycle)**:
   Ein Build-Lebenszyklus ist eine feste, vordefinierte Abfolge von einzelnen *Phasen* (Build Phases). Das Konzept sorgt dafür, dass jeder Entwickler und jedes automatisierte System (z. B. CI/CD-Pipelines) ein Projekt auf exakt dieselbe Weise baut.
   Die Phasen bauen aufeinander auf. Ruft man eine Phase auf, führt Maven automatisch alle vorhergehenden Phasen aus.

Phase	Fachliche Bedeutung und Aufgabe
`validate`	Prüft, ob das Projekt syntaktisch korrekt konfiguriert ist und alle notwendigen Informationen (z. B. in der `pom.xml`) vorhanden sind.
`compile`	Übersetzt den Java-Quellcode (`.java`) in Bytecode (`.class`) und legt diesen im Zielverzeichnis `target/classes/` ab.

Maven Grundlagen — Lösung (4/5)

4. **Der Maven Build-Lebenszyklus (Build Lifecycle)**:
   Ein Build-Lebenszyklus ist eine feste, vordefinierte Abfolge von einzelnen *Phasen* (Build Phases). Das Konzept sorgt dafür, dass jeder Entwickler und jedes automatisierte System (z. B. CI/CD-Pipelines) ein Projekt auf exakt dieselbe Weise baut.
   Die Phasen bauen aufeinander auf. Ruft man eine Phase auf, führt Maven automatisch alle vorhergehenden Phasen aus.

Phase	Fachliche Bedeutung und Aufgabe
`validate`	Prüft, ob das Projekt syntaktisch korrekt konfiguriert ist und alle notwendigen Informationen (z. B. in der `pom.xml`) vorhanden sind.
`compile`	Übersetzt den Java-Quellcode (`.java`) in Bytecode (`.class`) und legt diesen im Zielverzeichnis `target/classes/` ab.
`test`	

Maven Grundlagen — Lösung (4/5)

4. **Der Maven Build-Lebenszyklus (Build Lifecycle)**:
   Ein Build-Lebenszyklus ist eine feste, vordefinierte Abfolge von einzelnen *Phasen* (Build Phases). Das Konzept sorgt dafür, dass jeder Entwickler und jedes automatisierte System (z. B. CI/CD-Pipelines) ein Projekt auf exakt dieselbe Weise baut.
   Die Phasen bauen aufeinander auf. Ruft man eine Phase auf, führt Maven automatisch alle vorhergehenden Phasen aus.

Phase	Fachliche Bedeutung und Aufgabe
`validate`	Prüft, ob das Projekt syntaktisch korrekt konfiguriert ist und alle notwendigen Informationen (z. B. in der `pom.xml`) vorhanden sind.
`compile`	Übersetzt den Java-Quellcode (`.java`) in Bytecode (`.class`) und legt diesen im Zielverzeichnis `target/classes/` ab.
`test`	Führt automatisierte Unit-Tests (z. B. mit JUnit) aus. Schlagen Tests fehl, bricht der gesamte Build-Prozess an dieser Stelle ab.



Maven Grundlagen — Lösung (5/5)

4. **Der Maven Build-Lebenszyklus (Build Lifecycle - Fortsetzung):**

Phase	Fachliche Bedeutung und Aufgabe

Maven Grundlagen — Lösung (5/5)

4. **Der Maven Build-Lebenszyklus (Build Lifecycle - Fortsetzung):**

Phase	Fachliche Bedeutung und Aufgabe
package	

Maven Grundlagen — Lösung (5/5)

4. **Der Maven Build-Lebenszyklus (Build Lifecycle - Fortsetzung)**:

Phase	Fachliche Bedeutung und Aufgabe
package	Nimmt den kompilierten Bytecode und schnürt daraus das finale, auslieferbare Archiv-Format, das in der `pom.xml` definiert wurde (meist eine `.jar`- oder `.war`-Datei).

Maven Grundlagen — Lösung (5/5)

4. **Der Maven Build-Lebenszyklus (Build Lifecycle - Fortsetzung):**

Phase	Fachliche Bedeutung und Aufgabe
`package`	Nimmt den kompilierten Bytecode und schnürt daraus das finale, auslieferbare Archiv-Format, das in der `pom.xml` definiert wurde (meist eine `.jar`- oder `.war`-Datei).
`verify`	

Maven Grundlagen — Lösung (5/5)

4. **Der Maven Build-Lebenszyklus (Build Lifecycle - Fortsetzung):**

Phase	Fachliche Bedeutung und Aufgabe
`package`	Nimmt den kompilierten Bytecode und schnürt daraus das finale, auslieferbare Archiv-Format, das in der `pom.xml` definiert wurde (meist eine `.jar`- oder `.war`-Datei).
`verify`	Führt erweiterte Integrationstests aus und überprüft das erzeugte Paket auf Qualitätskriterien, um sicherzustellen, dass es stabil ist.

Maven Grundlagen — Lösung (5/5)

4. **Der Maven Build-Lebenszyklus (Build Lifecycle - Fortsetzung):**

Phase	Fachliche Bedeutung und Aufgabe
`package`	Nimmt den kompilierten Bytecode und schnürt daraus das finale, auslieferbare Archiv-Format, das in der `pom.xml` definiert wurde (meist eine `.jar`- oder `.war`-Datei).
`verify`	Führt erweiterte Integrationstests aus und überprüft das erzeugte Paket auf Qualitätskriterien, um sicherzustellen, dass es stabil ist.
`install`	

Maven Grundlagen — Lösung (5/5)

4. **Der Maven Build-Lebenszyklus (Build Lifecycle - Fortsetzung)**:

Phase	Fachliche Bedeutung und Aufgabe
`package`	Nimmt den kompilierten Bytecode und schnürt daraus das finale, auslieferbare Archiv-Format, das in der `pom.xml` definiert wurde (meist eine `.jar`- oder `.war`-Datei).
`verify`	Führt erweiterte Integrationstests aus und überprüft das erzeugte Paket auf Qualitätskriterien, um sicherzustellen, dass es stabil ist.
`install`	Kopiert das fertig geschnürte Paket (`.jar`) in das lokale Repository des Entwicklers. Dadurch kann dieses Artefakt als Abhängigkeit in anderen, eigenen lokalen Projekten verwendet werden.

Maven Grundlagen — Lösung (5/5)

4. **Der Maven Build-Lebenszyklus (Build Lifecycle - Fortsetzung):**

Phase	Fachliche Bedeutung und Aufgabe
`package`	Nimmt den kompilierten Bytecode und schnürt daraus das finale, auslieferbare Archiv-Format, das in der `pom.xml` definiert wurde (meist eine `.jar`- oder `.war`-Datei).
`verify`	Führt erweiterte Integrationstests aus und überprüft das erzeugte Paket auf Qualitätskriterien, um sicherzustellen, dass es stabil ist.
`install`	Kopiert das fertig geschnürte Paket (`.jar`) in das lokale Repository des Entwicklers. Dadurch kann dieses Artefakt als Abhängigkeit in anderen, eigenen lokalen Projekten verwendet werden.
`deploy`	

Maven Grundlagen — Lösung (5/5)

4. **Der Maven Build-Lebenszyklus (Build Lifecycle - Fortsetzung)**:

Phase	Fachliche Bedeutung und Aufgabe
`package`	Nimmt den kompilierten Bytecode und schnürt daraus das finale, auslieferbare Archiv-Format, das in der `pom.xml` definiert wurde (meist eine `.jar`- oder `.war`-Datei).
`verify`	Führt erweiterte Integrationstests aus und überprüft das erzeugte Paket auf Qualitätskriterien, um sicherzustellen, dass es stabil ist.
`install`	Kopiert das fertig geschnürte Paket (`.jar`) in das lokale Repository des Entwicklers. Dadurch kann dieses Artefakt als Abhängigkeit in anderen, eigenen lokalen Projekten verwendet werden.
`deploy`	Kopiert das finale Artefakt auf ein entferntes, zentrales Repository, um es anderen Entwicklern und Produktionsumgebungen zur Verfügung zu stellen.

Zusatz: Warum lernen wir Maven, wenn in Spring Boot alles „von selbst“ läuft?

Wer mit Spring Boot in einer modernen Entwicklungsumgebung wie *IntelliJ IDEA* arbeitet, kennt das: Man schreibt seinen Quellcode, klickt oben rechts auf das grüne Dreieck (Run-Button) und Sekunden später läuft das Backend. Von einem Werkzeug namens „Maven“ sieht man im Alltag auf den ersten Blick fast nichts.

Warum beschäftigen wir uns also mit Maven-Koordinaten und Build-Lebenszyklen? Die Antwort ist einfach: **Maven ist der unsichtbare Architekt Ihres Projekts.**

1. **Das „grüne Dreieck“ ist eine Abkürzung**(1/2) Wenn Sie in IntelliJ auf den Start-Button klicken, passiert im Hintergrund Folgendes:
   - Die IDE schaut in die Datei `pom.xml`, um zu prüfen, welche Java-Version und welche externen Bibliotheken (z. B. *Spring Web*, *Spring Data JPA*) Sie für Ihren Code angefordert haben.
   - IntelliJ nutzt im Hintergrund die Logik der Maven-Phase `compile`, um Ihre geschriebenen `.java`-Dateien in ausführbaren Bytecode (`.class`) zu übersetzen.
   - Danach startet die IDE die Anwendung sofort direkt aus diesen losen, frisch übersetzten Dateien.

Zusatz: Warum lernen wir Maven, wenn in Spring Boot alles „von selbst“ läuft?

1. **Das „grüne Dreieck“ ist eine Abkürzung**(2/2) Das ist extrem praktisch, weil es beim Programmieren viel Zeit spart. Es führt jedoch oft zu dem Trugschluss, dass man Maven erst ganz am Ende für das spätere „Deployment“ (den Serverbetrieb) benötigt. In Wahrheit nutzt IntelliJ die von Maven definierte Struktur in jeder Sekunde der Entwicklung.

Zusatz: Warum lernen wir Maven, wenn in Spring Boot alles „von selbst“ läuft?

2. **Der Lifecycle: Ihr Sicherheitsgurt für die Praxis**

   Während des Programmierens interessiert uns tatsächlich meistens nur die Phase `compile`. Doch der eigentliche Maven-Lebenszyklus (*Build Lifecycle*) ist eine feste Kette von Schritten, die strikt aufeinander aufbauen. Und diese Kette wird genau dann lebenswichtig, wenn wir die schützende und automatisierte Umgebung unserer IDE verlassen.

   Sobald ein Projekt fertiggestellt, im Team geteilt oder in eine CI/CD-Pipeline zur Server-Bereitstellung übergeben wird, entfaltet Maven seine volle Kontrollfunktion und erzwingt den Standard-Lebenszyklus von vorne bis hinten:

   - `validate` & `compile`: Der Quellcode wird formal auf syntaktische Korrektheit geprüft und komplett frisch übersetzt.
   - `test`: Maven sucht nach *allen* im Projekt hinterlegten automatisierten Tests (z. B. JUnit) und führt diese aus. Schlägt auch nur ein einziger Test fehl, bricht Maven den Prozess sofort ab. Dies ist ein genialer Schutzmechanismus, damit niemals fehlerhafter Code in Produktion geht.

Zusatz: Warum lernen wir Maven, wenn in Spring Boot alles „von selbst“ läuft?

- `package`: Sind alle Tests grün, schnürt Maven das finale Paket – eine sogenannte „Fat JAR“. Diese einzelne Datei enthält Ihren gesamten Code, die statischen Frontend-Dateien und sogar den kompletten Webserver (*Tomcat*).
- `install` & `deploy`: Zum Schluss wird das fertige, geprüfte Paket entweder in das lokale Repository kopiert (`install`), damit andere Entwickler im Team es nutzen können, oder direkt auf einen echten Webserver bzw. in ein zentrales Firmen-Repository hochgeladen (`deploy`).

Das Fazit für die Praxis: In der täglichen Entwicklung nimmt uns die IDE die Arbeit ab und nutzt nur das absolute Minimum von Maven (`compile`). Erst der vollständige Maven-Lebenszyklus bis hin zu `package` oder `deploy` sorgt jedoch dafür, dass Ihr Projekt völlig unabhängig von IntelliJ, Betriebssystemen oder Cloud-Anbietern auf jedem Rechner der Welt exakt gleich getestet, gebaut und gestartet werden kann.

Session Tracking Vergleich

Session Tracking Vergleich

Vergleichen Sie die fünf gängigen Session-Tracking-Methoden. Füllen Sie dazu die folgende Tabelle aus:

Methode	Funktioniert ohne Cookies?	XSS-Risiko?	Typischer Einsatz
URL-Kodierung	?	?	?
Hidden Fields	?	?	?
Cookies	?	?	?
HTTP Authorization Header	?	?	?
Session Tracking API (`HttpSession`)	?	?	?

Session Tracking Vergleich — Lösung (1/2)

Methode	Ohne Cookies?	XSS-Risiko?	Typischer Einsatz
URL-Kodierung			
Hidden Fields			
Cookies			
HTTP Authorization Header			
Session Tracking API (`HttpSession`)			

Session Tracking Vergleich — Lösung (1/2)

Methode	Ohne Cookies?	XSS-Risiko?	Typischer Einsatz
URL-Kodierung	Ja		
Hidden Fields			
Cookies			
HTTP Authorization Header			
Session Tracking API (`HttpSession`)			

Session Tracking Vergleich — Lösung (1/2)

Methode	Ohne Cookies?	XSS-Risiko?	Typischer Einsatz
URL-Kodierung	Ja	Hoch	
Hidden Fields			
Cookies			
HTTP Authorization Header			
Session Tracking API (`HttpSession`)			

Session Tracking Vergleich — Lösung (1/2)

Methode	Ohne Cookies?	XSS-Risiko?	Typischer Einsatz
URL-Kodierung	Ja	Hoch	Fallback, wenn Cookies deaktiviert sind
Hidden Fields			
Cookies			
HTTP Authorization Header			
Session Tracking API (`HttpSession`)			

Session Tracking Vergleich — Lösung (1/2)

Methode	Ohne Cookies?	XSS-Risiko?	Typischer Einsatz
URL-Kodierung	Ja	Hoch	Fallback, wenn Cookies deaktiviert sind
Hidden Fields	Ja		
Cookies			
HTTP Authorization Header			
Session Tracking API (`HttpSession`)			

Session Tracking Vergleich — Lösung (1/2)

Methode	Ohne Cookies?	XSS-Risiko?	Typischer Einsatz
URL-Kodierung	Ja	Hoch	Fallback, wenn Cookies deaktiviert sind
Hidden Fields	Ja	Hoch	
Cookies			
HTTP Authorization Header			
Session Tracking API (`HttpSession`)			

Session Tracking Vergleich — Lösung (1/2)

Methode	Ohne Cookies?	XSS-Risiko?	Typischer Einsatz
URL-Kodierung	Ja	Hoch	Fallback, wenn Cookies deaktiviert sind
Hidden Fields	Ja	Hoch	Mehrstufige Formulare
Cookies			
HTTP Authorization Header			
Session Tracking API (`HttpSession`)			

Session Tracking Vergleich — Lösung (1/2)

Methode	Ohne Cookies?	XSS-Risiko?	Typischer Einsatz
URL-Kodierung	Ja	Hoch	Fallback, wenn Cookies deaktiviert sind
Hidden Fields	Ja	Hoch	Mehrstufige Formulare
Cookies	Nein		
HTTP Authorization Header			
Session Tracking API (`HttpSession`)			

Session Tracking Vergleich — Lösung (1/2)

Methode	Ohne Cookies?	XSS-Risiko?	Typischer Einsatz
URL-Kodierung	Ja	Hoch	Fallback, wenn Cookies deaktiviert sind
Hidden Fields	Ja	Hoch	Mehrstufige Formulare
Cookies	Nein	Gering (mit `HttpOnly`)	
HTTP Authorization Header			
Session Tracking API (`HttpSession`)			

Session Tracking Vergleich — Lösung (1/2)

Methode	Ohne Cookies?	XSS-Risiko?	Typischer Einsatz
URL-Kodierung	Ja	Hoch	Fallback, wenn Cookies deaktiviert sind
Hidden Fields	Ja	Hoch	Mehrstufige Formulare
Cookies	Nein	Gering (mit `HttpOnly`)	Standard für klassische Webanwendungen
HTTP Authorization Header			
Session Tracking API (`HttpSession`)			

Session Tracking Vergleich — Lösung (1/2)

Methode	Ohne Cookies?	XSS-Risiko?	Typischer Einsatz
URL-Kodierung	Ja	Hoch	Fallback, wenn Cookies deaktiviert sind
Hidden Fields	Ja	Hoch	Mehrstufige Formulare
Cookies	Nein	Gering (mit `HttpOnly`)	Standard für klassische Webanwendungen
HTTP Authorization Header	Ja		
Session Tracking API (`HttpSession`)			

Session Tracking Vergleich — Lösung (1/2)

Methode	Ohne Cookies?	XSS-Risiko?	Typischer Einsatz
URL-Kodierung	Ja	Hoch	Fallback, wenn Cookies deaktiviert sind
Hidden Fields	Ja	Hoch	Mehrstufige Formulare
Cookies	Nein	Gering (mit `HttpOnly`)	Standard für klassische Webanwendungen
HTTP Authorization Header	Ja	gering, Abhängig von der Token-Speicherung,	
Session Tracking API (`HttpSession`)			

Session Tracking Vergleich — Lösung (1/2)

Methode	Ohne Cookies?	XSS-Risiko?	Typischer Einsatz
URL-Kodierung	Ja	Hoch	Fallback, wenn Cookies deaktiviert sind
Hidden Fields	Ja	Hoch	Mehrstufige Formulare
Cookies	Nein	Gering (mit `HttpOnly`)	Standard für klassische Webanwendungen
HTTP Authorization Header	Ja	gering, Abhängig von der Token-Speicherung,	REST-APIs, Single-Page-Apps (JWT)
Session Tracking API (`HttpSession`)			

Session Tracking Vergleich — Lösung (1/2)

Methode	Ohne Cookies?	XSS-Risiko?	Typischer Einsatz
URL-Kodierung	Ja	Hoch	Fallback, wenn Cookies deaktiviert sind
Hidden Fields	Ja	Hoch	Mehrstufige Formulare
Cookies	Nein	Gering (mit `HttpOnly`)	Standard für klassische Webanwendungen
HTTP Authorization Header	Ja	gering, Abhängig von der Token-Speicherung,	REST-APIs, Single-Page-Apps (JWT)
Session Tracking API (`HttpSession`)	Ja (via Fallback)		



Session Tracking Vergleich — Lösung (1/2)

Methode	Ohne Cookies?	XSS-Risiko?	Typischer Einsatz
URL-Kodierung	Ja	Hoch	Fallback, wenn Cookies deaktiviert sind
Hidden Fields	Ja	Hoch	Mehrstufige Formulare
Cookies	Nein	Gering (mit `HttpOnly`)	Standard für klassische Webanwendungen
HTTP Authorization Header	Ja	gering, Abhängig von der Token-Speicherung,	REST-APIs, Single-Page-Apps (JWT)
Session Tracking API (`HttpSession`)	Ja (via Fallback)	Gering (durch Framework-Schutz)	



Session Tracking Vergleich — Lösung (1/2)

Methode	Ohne Cookies?	XSS-Risiko?	Typischer Einsatz
URL-Kodierung	Ja	Hoch	Fallback, wenn Cookies deaktiviert sind
Hidden Fields	Ja	Hoch	Mehrstufige Formulare
Cookies	Nein	Gering (mit `HttpOnly`)	Standard für klassische Webanwendungen
HTTP Authorization Header	Ja	gering, Abhängig von der Token-Speicherung,	REST-APIs, Single-Page-Apps (JWT)
Session Tracking API (`HttpSession`)	Ja (via Fallback)	Gering (durch Framework-Schutz)	Serverzentrierte Java-Webapplikationen



Session Tracking Vergleich — Lösung (2/2)

Zusatz: Erklärungen der Technologien und Tabelleneinträge:

- **Cross-Site Scripting (XSS)**: Cross-Site Scripting bezeichnet eine Schwachstelle, bei der Angreifer bösartigen Client-seitigen Code (meist JavaScript) in eine vertrauenswürdige Webseite einschleusen. Da der Browser des Opfers diesen Code unhinterfragt im Kontext der aktuellen Sitzung ausführt, können Angreifer dadurch sensible Daten wie Session-IDs auslesen oder unautorisierte Aktionen im Namen des Nutzers durchführen.
- **URL-Kodierung**: Die Session-ID wird als Pfadparameter oder Query-String direkt an jede URL angehängt (z. B. `;jsessionid=123`).
- **Hidden Fields**: Die ID wird in einem unsichtbaren HTML-Eingabefeld (`<input type="hidden">`) innerhalb eines Formulars transportiert.
- **Cookies**: Der Server sendet ein `Set-Cookie`-Header, woraufhin der Browser die Session-ID bei jedem Folge-Request automatisch im Header mitsendet.
- **HTTP Authorization Header**: Der HTTP Authorization Header ist ein standardisiertes Feld im Kopfbereich (Header) einer HTTP-Anfrage, mit dem ein Client Authentifizierungsdaten an den Server übermittelt. Hierbei übergibt der Client die Session- oder Token-Information direkt und explizit im Headerfeld Authorization, beispielsweise in Form eines Bearer Tokens. Technisch gesehen läuft diese Methode völlig losgelöst vom Cookie-Mechanismus des Browsers.
- **Session Tracking API** (`HttpSession`): Dies ist das serverseitige High-Level-Interface von Jakarta/Java EE zur Sitzungsverwaltung.



Spring Boot REST-Controller



Spring Boot REST-Controller (1/2)

Implementieren Sie einen REST-Controller für ein einfaches Studierenden-Verwaltungssystem. Der Controller soll Anfragen annehmen und über ein bereitgestelltes Repository (`StudentRepository`) Daten abfragen oder verändern.

Das Projekt *studierendenverwaltung_vorlage* ist in Stud.IP hinterlegt. Dieses enthält bereits das Datenmodell `Student` und ein simuliertes `StudentRepository` (mit bereits hinterlegten Testdaten).

Spring Boot REST-Controller (2/2)

Erweitern Sie ausschließlich die vorgegebene Controller-Klasse `StudentController`, sodass die folgenden HTTP-Endpunkte unterstützt werden:

- `GET /students` — Gibt eine Liste aller Studierenden zurück (HTTP Status 200 OK).
- `GET /students/{id}` — Sucht einen Studierenden nach seiner ID. Wenn er existiert, wird er zurückgegeben (200 OK). Wenn nicht, soll der HTTP-Status 404 Not Found geliefert werden.
- `POST /students` — Legt einen neuen Studierenden an (201 Created). Der Endpunkt erwartet die Daten im JSON-Format im Request-Body. Reichen Sie im HTTP-Response-Header `Location` die URI des neuen Studierenden mit zurück (z. B. `/students/3`).
- `DELETE /students/{id}` — Löscht den Studierenden mit der angegebenen ID aus dem System (204 No Content).

Spring Boot REST-Controller — Lösung (1/5)

Implementierung Controller:

```java
package wde.studierendenverwaltung;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.net.URI;
import java.util.List;
import java.util.Optional;

@RestController // Kennzeichnet die Klasse als REST-Controller (gibt JSON statt HTML zurueck)
@RequestMapping("/students") // Alle Endpunkte in dieser Klasse starten mit /students
public class StudentController {

  // Das Repository wird per Constructor Injection automatisch von Spring bereitgestellt
  private final StudentRepository repo;

  public StudentController(StudentRepository repo) {
    this.repo = repo;
  }
```

Spring Boot REST-Controller — Lösung (2/5)

Implementierung Controller:

```
// 1. GET ALL: Alle Studierenden abrufen
@GetMapping("")
public ResponseEntity<List<Student>> getAll() {
  // Status 200 OK und die Studierenden im Body
  return ResponseEntity.ok(repo.findAll());
}
```



Spring Boot REST-Controller — Lösung (3/5)

```
// 2. GET BY ID: Einzelnen Studierenden suchen
@GetMapping("/{id}")
public ResponseEntity<Student> getById(@PathVariable Long id) {
  // Suchen im Repository liefert ein Optional (kann leer sein)
  Optional<Student> studentOpt = repo.findById(id);

  if (studentOpt.isPresent()) {
    // Wenn gefunden: Status 200 OK und das Student-Objekt im Body
    return ResponseEntity.ok(studentOpt.get());
  } else {
    // Wenn nicht gefunden: Status 404 Not Found
    return ResponseEntity.notFound().build();
  }
}
```



Spring Boot REST-Controller — Lösung (4/5)

```java
// 3. POST: Neuen Studierenden anlegen
@PostMapping("")
public ResponseEntity<Student> create(@RequestBody Student student) {
  // @RequestBody liest das JSON aus dem Request und baut daraus das Student-Objekt
  Student savedStudent = repo.save(student);

  // Erstellen der Location-URI fuer den Response-Header (z.B. /students/3)
  URI location = URI.create("/students/" + savedStudent.getId());

  // Status 201 Created zurueckgeben, inklusive Location-Header und dem gespeicherten Objekt
  return ResponseEntity.created(location).body(savedStudent);
}
```



Spring Boot REST-Controller — Lösung (5/5)

```
  // 4. DELETE: Studierenden loeschen
  @DeleteMapping("/{id}")
  public ResponseEntity<Void> delete(@PathVariable Long id) {
    // @PathVariable liest die ID aus der URL
    repo.deleteById(id);

    // Status 204 No Content (Erfolgreich geloescht, keine Daten im Body)
    return ResponseEntity.noContent().build();
  }
}
```



MVC, MVP und MVVM im Vergleich

MVC, MVP und MVVM im Vergleich

1. Erklären Sie die Begriffe MVC, MVP und MVVM jeweils in ein bis zwei Sätzen.

2. Füllen Sie die folgende Tabelle aus:

Aspekt	MVC	MVP	MVVM
Wer kommuniziert mit dem Model?	?	?	?
Reaktion auf Änderungen	?	?	?
Testbarkeit	?	?	?
Typische Plattformen/Frameworks	?	?	?

3. Warum gilt das MVP-Muster als besonders testfreundlich?

MVC, MVP und MVVM im Vergleich — Lösung (1/3)

1. Erklärungen der Begriffe:
   -
   -
   -

MVC, MVP und MVVM im Vergleich — Lösung (1/3)

1. Erklärungen der Begriffe:
   - **MVC (Model–View–Controller)**: Das Model bildet die zentrale Komponente des Patterns und übernimmt das Management der Daten sowie der Geschäftslogik und Regeln der Anwendung. Die View ist eine visuelle Repräsentation der Informationen etwa in Form von Tabellen, Diagrammen o. ä., wobei mehrere Views für dieselben Informationen existieren können. Der Controller nimmt Eingaben entgegen und verarbeitet diese zu Anweisungen für das Model und die View.
   -
   -

MVC, MVP und MVVM im Vergleich — Lösung (1/3)

1. Erklärungen der Begriffe:
   - **MVC (Model–View–Controller)**: Das Model bildet die zentrale Komponente des Patterns und übernimmt das Management der Daten sowie der Geschäftslogik und Regeln der Anwendung. Die View ist eine visuelle Repräsentation der Informationen etwa in Form von Tabellen, Diagrammen o. ä., wobei mehrere Views für dieselben Informationen existieren können. Der Controller nimmt Eingaben entgegen und verarbeitet diese zu Anweisungen für das Model und die View.
   - **MVP (Model–View–Presenter)**: Der Presenter wird zwischen Model und View geschaltet und übernimmt die Kommunikation zwischen beiden Komponenten. Die View ist möglichst passiv und delegiert Benutzeraktionen an den Presenter. Der Presenter enthält die Präsentationslogik und aktualisiert die View über ein Interface.
   -

MVC, MVP und MVVM im Vergleich — Lösung (1/3)

1. Erklärungen der Begriffe:
   - **MVC (Model–View–Controller)**: Das Model bildet die zentrale Komponente des Patterns und übernimmt das Management der Daten sowie der Geschäftslogik und Regeln der Anwendung. Die View ist eine visuelle Repräsentation der Informationen etwa in Form von Tabellen, Diagrammen o. ä., wobei mehrere Views für dieselben Informationen existieren können. Der Controller nimmt Eingaben entgegen und verarbeitet diese zu Anweisungen für das Model und die View.
   - **MVP (Model–View–Presenter)**: Der Presenter wird zwischen Model und View geschaltet und übernimmt die Kommunikation zwischen beiden Komponenten. Die View ist möglichst passiv und delegiert Benutzeraktionen an den Presenter. Der Presenter enthält die Präsentationslogik und aktualisiert die View über ein Interface.
   - **MVVM (Model–View–ViewModel)**: Das ViewModel stellt Daten und Operationen für die View bereit. Es hält den aktuellen Zustand der View und transformiert die Daten des Models in das von der View benötigte Format. Die Synchronisation zwischen View und ViewModel erfolgt häufig über Data Binding.

MVC, MVP und MVVM im Vergleich — Lösung (2/3)

b) Vergleichstabelle:

Aspekt	MVC	MVP	MVVM
Wer kommuniziert mit dem Model?	Controller und View	Presenter	ViewModel
Reaktion auf Änderungen	Controller stößt Rendering an	Presenter aktualisiert View	Data Binding
Testbarkeit	gut	sehr gut	gut
Typische Plattformen/Frameworks	Spring MVC	Android (klassisch)	Angular, Vue

MVC, MVP und MVVM im Vergleich — Lösung (3/3)

c) **Testfreundlichkeit von MVP**:

MVP ist besonders testfreundlich, da die View lediglich über ein Interface angesprochen wird und der Presenter die Logik enthält. Der Presenter kann dadurch unabhängig von konkreten UI-Komponenten mit Mock-Objekten getestet werden.

Materialien zum Übungsblatt

- Aufgabe *Spring Boot REST Controller*: Vorlagendateien Studierendenverwaltung (`studierendenverwaltung_vorlage.zip`)

Lösungen:

- Aufgabe *Spring Boot REST Controller*:Lösung Studierendenverwaltung (`studierendenverwaltung_lsg.zip`)